Static Maze Solver
(Search vs. Optimization)
Comparison Between Breadth First Search (BFS) and Genetic Algorithm (GA)

Prepared By :

Shahd Mohamed Abdo

Mariam Osama Hassan

Ashraqat Sabra Mohamed

Faculty of Computers and Information
Department of Artificial Intelligence
Second Year

Supervised By:

Dr. Amal

8 May , 2026

1.0 Introduction

This project compares a traditional search algorithm (BFS) with a Genetic Algorithm (GA) in solving the same 2D maze problem.

The maze is represented using a NumPy array where:

- 0 represents a walkable path.
- 1 represents a wall.

The start point is fixed at (0,0) and the goal point is fixed at (9,9).

The purpose of this project is to compare:

- Accuracy
- Optimality
- Performance
  between classical search methods and optimization algorithms.

2.0 Environment Design

The environment is a fixed 10×10 maze represented as a 2D NumPy array.

Each cell contains:

- 0 → Walkable cell
- 1 → Wall

The agent starts from the top-left corner and attempts to reach the bottom-right corner while avoiding walls and invalid moves.

3.0 Search Algorithm (BFS)

The project uses Breadth First Search (BFS) as the search algorithm.

BFS explores nodes level by level and guarantees finding the shortest path in an unweighted maze.

We selected BFS because :

- It guarantees the optimal shortest path.
- It is complete and systematic.
- It performs well in static grid environments.

BFS vs DFS

BFS	DFS
Finds the shortest path	May not find the shortest path
Uses more memory	Uses less memory
Better for maze solving	Can get trapped in deep paths
Complete and systematic	Explores one branch deeply first
More suitable for unweighted mazes	Faster in some simple cases

Therefore, BFS was more suitable for this project.

4.0 Chromosome Representation

In the Genetic Algorithm , each chromosome represents a sequence of moves.

Each gene is an integer :

- 0 → Up
- 1 → Right
- 2 → Down
- 3 → Left

Example :

[1,1,2,2,3,0]

This means :
Right → Right → Down → Down → Left → Up

The chromosome length was set to 40 moves to give the agent enough steps to potentially reach the goal.

5.0 Fitness Function

The fitness function evaluates how close the agent gets to the goal.

The Manhattan Distance was used :

$$distance = | x_{goal} - x | + | y_{goal} - y |$$

The fitness equation is :

$$fitness = \frac{100}{1 + distance} - 2(invalid_moves) + bonus$$

Where :

- Smaller distance increases fitness.
- Invalid moves decrease fitness.
- A large bonus (+1000) is added if the goal is reached.

This fitness function encourages :

- Moving closer to the goal.
- Avoiding walls.
- Reaching the goal successfully.

6.0 Selection Algorithm

Tournament Selection was used.

In this method :

- A small random group of chromosomes is selected.
- The chromosome with the best fitness is chosen as a parent.

This method was selected because :

- It is simple and efficient.
- It maintains competition between solutions.
- It improves convergence speed.

7.0 Crossover

Single-Point Crossover was used.

A random crossover point is selected, and the two parents exchange genes after that point to generate offspring.

This method was suitable because :

- It preserves useful move sequences.
- It creates diversity in the population.
- It is simple to implement.

8.0 Mutation

Random Mutation was used with a mutation rate of 0.05.

Each gene has a 5% probability of changing into another random direction.

Mutation helps :

- Prevent premature convergence.
- Increase population diversity.
- Explore new possible solutions.

Random Mutation was suitable for this project because the maze problem requires exploration of many possible movement combinations.

By randomly changing directions inside the chromosome, the algorithm can discover new paths that may lead closer to the goal.

It also helps prevent the Genetic Algorithm from getting stuck in poor local solutions and increases population diversity.

9.0 Termination Condition

The Genetic Algorithm stops when:

- The maximum number of generations is reached.
- The best fitness value does not improve for 5 consecutive generations, indicating convergence.
- The agent successfully reaches the target (goal state), which represents an optimal solution.

10.0 Visualization

Matplotlib was used to visualize the maze and paths.

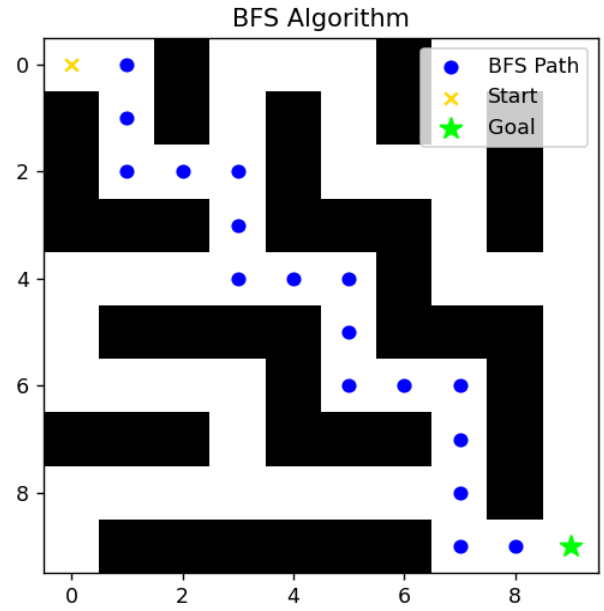
Visualization details :

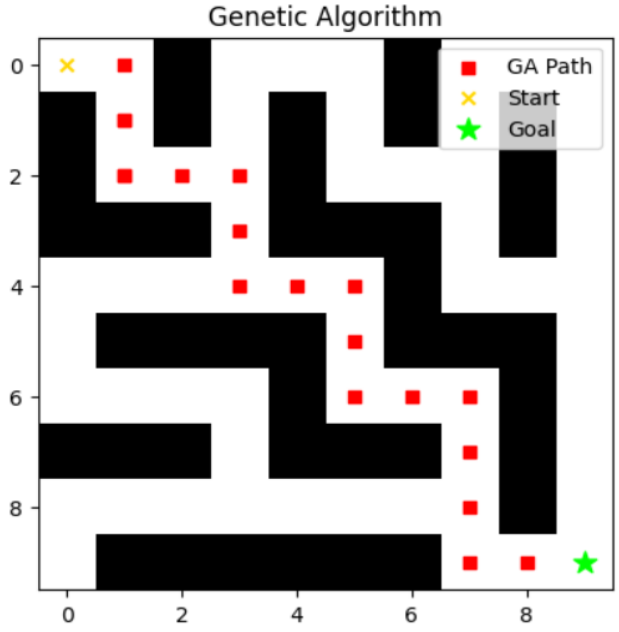
- White cells → Walkable paths
- Black cells → Walls
- Blue circles → BFS path
- Red squares → GA path

- yellow X → Start point
- Green star → Goal point

The visualization helps compare :

- Path quality
- Accuracy
- Similarities and differences between BFS and GA





11.0 Comparison Between BFS and GA

BFS	Genetic Algorithm
Finds shortest path	Does not guarantee shortest path
Deterministic	Probabilistic
Faster in static mazes	Slower due to generations
Exact solution	Approximate optimization

BFS produced the optimal shortest path , while the Genetic Algorithm produced a near-optimal path using evolutionary optimization techniques.

12.0 Team Roles

- Mariam Osama Hassan : Implemented visualization
- Ashraqat Sabra Mohamed : Implemented BFS
- Shahd Mohamed Abdo : report writing
- All team members : contributed to environment design and testing the Genetic Algorithm operators

13.0 Conclusion

This project demonstrated the difference between classical search algorithms and optimization algorithms in solving maze problems.

BFS successfully found the shortest path efficiently, while the Genetic Algorithm demonstrated how evolutionary computation can approximate solutions through selection, crossover, and mutation.